

# CONTENTS

## M4 — Ejercicios Adicionales de Práctica

### Máquinas de Estado, Síntesis y Análisis de Circuitos Secuenciales

#### Índice de dificultad

#### Ejercicio 1 ★★ — FSM Moore: detector de secuencia “011” (con solapamiento)

##### Paso 1 — Tabla de estados

##### Paso 2 — Codificación

##### Paso 3 — Tabla de transiciones y salida (Moore: $Y$ es función solo del estado)

##### Paso 4 y 5 — Ecuaciones de excitación ( $FF D \rightarrow D_i = \text{valor deseado de } q_i \text{ directamente}$ ) y Karnaugh

##### Paso 6 — Circuito

##### ✓ Verificación

#### Ejercicio 2 ★ — FSM Moore: contador módulo 6 con indicador de múltiplo de 3

##### Paso 1 — Tabla de estados (3 bits, $q_2q_1q_0$ , códigos 6 y 7 no se usan)

##### Paso 2 y 3 — Tabla de excitación T ( $T=1$ exactamente cuando el bit cambia entre estado y próximo estado)

##### Paso 4 y 5 — Ecuaciones minimizadas (Karnaugh con don't-cares 110,111)

##### Paso 6 — Circuito

##### ✓ Verificación

#### Ejercicio 3 ★★ — FSM Mealy: mismo detector de “011” con solapamiento (comparar con el Ejercicio 1)

##### Paso 1 — Tabla de estados (Mealy necesita un estado menos)

##### Paso 2 y 3 — Codificación ( $A=00$ , $B=01$ , $C=10$ ; código 11 no se usa) y tabla de transiciones

##### Paso 4 y 5 — Ecuaciones (Karnaugh de 3 variables, con don't-care en $q_1q_0=11$ )

##### Paso 6 — Circuito

##### ✓ Verificación

#### Ejercicio 4 ★★★ — FSM Mealy: detector de “1001” sin solapamiento

Paso 1 — Tabla de estados

Paso 2 y 3 — Codificación ( $S_0=00$ ,  $S_1=01$ ,  $S_2=10$ ,  $S_3=11$ ) y tabla de transiciones

Paso 4 y 5 — Ecuaciones (Karnaugh de 3 variables, sin don't-cares — se usan los 4 estados)

Paso 6 — Circuito

☒ Verificación

#### Ejercicio 5 ★★★ — Síntesis completa con FF JK: contador BCD (0 a 9)

Paso 1 — Tabla de transiciones (10 estados válidos; 6 códigos no usados → don't care)

Paso 2 — Tabla de excitación JK (aplicando la tabla de excitación de la Guía M4 sección 5.2 a cada bit, fila por fila)

Paso 3 y 4 — Karnaugh de 4 variables para cada  $J_i, K_i$  (aprovechando todos los X como comodines)

Paso 5 — Circuito

☒ Verificación

#### Ejercicio 6 ★★ — Síntesis completa con FF D: tabla de transición dada directamente

Paso 1 — Ecuaciones de excitación (FF D:  $D_i = \text{próximo valor de } q_i$  directamente, se toma tal cual de la tabla)

Paso 2 — Karnaugh de 3 variables ( $q_1, q_0, x$ ) para  $D_1, D_0$  e  $Y$

Paso 3 — Circuito

☒ Verificación

#### Ejercicio 7 ★★★ — Análisis: dado el circuito, hallar la tabla de estados y el diagrama

Paso 1 — Evaluar  $J, K$  para las 8 combinaciones de  $(q_1, q_0, x)$

Paso 2 — Tabla de estados equivalente (renombrando  $S_0=A, S_1=B, S_2=C, S_3=D$ )

Paso 3 — Diagrama de transiciones (descripción) y observación de alcanzabilidad

☒ Verificación

Preguntas teóricas rápidas tipo parcial

Resumen ultra-rápido de este documento



# M4 — Ejercicios Adicionales de Práctica

## MÁQUINAS DE ESTADO, SÍNTESIS Y ANÁLISIS DE CIRCUITOS SECUENCIALES

Arquitectura de Computadoras — FING UdelaR — Módulo 4

Estos son 7 ejercicios **nuevos**, distintos de los del Práctico 5 (que detecta “1101” con Mealy, contador up/down con FF T, paridad, comparador serial, código Gray, contador de primos y el cronómetro) y distintos de los ejemplos de la Guía Temática (que usa “1101” Mealy como ejemplo guía, contador binario módulo 4, up/down de 3 bits, carga paralela y Gray de 3 bits). Acá se practican los mismos subtemas con especificaciones nuevas: otra secuencia a detectar, otro tipo de FF, un contador BCD, una tabla dada directamente, y un análisis inverso (circuito → tabla → diagrama).

**Verificación:** cada tabla de transición y cada ecuación de excitación de este documento se generó y comprobó con un script de Python (`sympy.logic.boolalg.SOPform`) para la minimización por fuerza bruta a partir de los minterminos — equivalente a un Karnaugh exhaustivo, incluyendo don't-cares donde corresponde — más simulación bit a bit contra la especificación en lenguaje natural para las máquinas de estado, con miles de cadenas de entrada aleatorias). El script queda documentado dentro de cada ejercicio en la sección  Verificación.

### Índice de dificultad

| # | Ejercicio                                                | Tipo              | FF | Dificultad        |
|---|----------------------------------------------------------|-------------------|----|-------------------|
| 1 | Detector de “011” con solapamiento                       | FSM Moore         | D  | ★★ Medio          |
| 2 | Contador módulo 6 con indicador de múltiplo de 3         | FSM Moore         | T  | ★ Fácil-Medio     |
| 3 | Detector de “011” con solapamiento (versión Mealy de #1) | FSM Mealy         | D  | ★★ Medio          |
| 4 | Detector de “1001” sin solapamiento                      | FSM Mealy         | D  | ★★★ Medio-Difícil |
| 5 | Contador BCD (0 a 9)                                     | Síntesis completa | JK | ★★★ Difícil       |
| 6 | Tabla de transición abstracta dada                       | Síntesis completa | D  | ★★ Medio          |
| 7 | Circuito con 2 FF JK dado — análisis inverso             | Análisis          | JK | ★★★ Difícil       |

### Ejercicio 1 ★★ — FSM Moore: detector de secuencia “011” (con solapamiento)

**Enunciado:** diseñar, con Flip-Flops D, un circuito con una entrada serial  $x$  y una salida  $Y$  tal que  $Y=1$  exactamente mientras el circuito se encuentre en el estado alcanzado inmediatamente después de haber recibido los tres últimos bits “0,1,1” (se permite solapamiento: por ejemplo, con la entrada 0,1,1,0,1,1 debe indicar detección dos veces). Usar el enfoque de **máquina de Moore** (la salida debe depender solo del estado).

**PASO 1 — TABLA DE ESTADOS**

- **S0**: no hay ningún prefijo útil de “011” reconocido.
- **S1**: el último bit relevante fue “0” (posible comienzo).
- **S2**: se vieron “01”.
- **S3**: se completó “011” —  $Y=1$  en este estado.

| Estado | $x=0 \rightarrow$ próximo | $x=1 \rightarrow$ próximo |
|--------|---------------------------|---------------------------|
| S0     | S1                        | S0                        |
| S1     | S1                        | S2                        |
| S2     | S1                        | S3                        |
| S3     | S1                        | S0                        |

Razonamiento de las dos filas más delicadas (las que se saltan bajo presión de tiempo): en **S2** (visto “01”), si llega otro 1 se forma “011”  $\rightarrow$  **S3**. En **S3** ( $Y=1$ , se acaba de completar “011”), si llega un 0 el sufijo relevante es “0”  $\rightarrow$  vuelve a **S1** (posible comienzo solapado); si llega un 1, el sufijo son los dos últimos bits “11”, que **no** es prefijo de “011” (que empieza con 0)  $\rightarrow$  vuelve a **S0**.

**PASO 2 — CODIFICACIÓN**

4 estados  $\rightarrow$  2 FF ( $q_1, q_0$ ).  $S0=00$ ,  $S1=01$ ,  $S2=10$ ,  $S3=11$ .

**PASO 3 — TABLA DE TRANSICIONES Y SALIDA (MOORE:  $Y$  ES FUNCIÓN SOLO DEL ESTADO)**

| Estado | $q_1q_0$ | $x$ | próximo | $D_1D_0$ | $Y$ (del estado actual) |
|--------|----------|-----|---------|----------|-------------------------|
| S0     | 00       | 0   | S1      | 01       | 0                       |
| S0     | 00       | 1   | S0      | 00       | 0                       |
| S1     | 01       | 0   | S1      | 01       | 0                       |
| S1     | 01       | 1   | S2      | 10       | 0                       |
| S2     | 10       | 0   | S1      | 01       | 0                       |
| S2     | 10       | 1   | S3      | 11       | 0                       |
| S3     | 11       | 0   | S1      | 01       | 1                       |
| S3     | 11       | 1   | S0      | 00       | 1                       |

**PASO 4 Y 5 — ECUACIONES DE EXCITACIÓN (FF D  $\rightarrow$   $D_i$  = VALOR DESEADO DE  $Q_i$  DIRECTAMENTE) Y KARNAUGH**

Minimizando por Karnaugh de 3 variables ( $q_1, q_0, x$ ):

$$D_1 = q_1 \cdot q_0' + q_0 \cdot q_1' \quad D_0 = x' + q_1 \cdot q_0' \quad Y = q_1 \cdot q_0$$

## PASO 6 — CIRCUITO

2 FF D.  $D_1$ : dos AND de 2 entradas (una con  $q_1, \overline{q_0}$ ... en realidad 3 literales cada término) más un OR;  $D_0$ : un NOT de  $x$ , un AND ( $\overline{q_1} \cdot q_0$ ) y un OR.  $Y$ : un solo AND de 2 entradas ( $q_1 \cdot q_0$ ), tomado directamente de las salidas de los FF (no hace falta reloj adicional, es Moore puro).

### ✓ VERIFICACIÓN

Script Python: se construyeron las 8 filas de la tabla de transición a partir de la especificación en lenguaje natural, se generaron  $D_1, D_0$  con `SOPform` a partir de sus minitérminos y se comprobó que, evaluados en las 8 combinaciones de  $(q_1, q_0, x)$ , reproducen exactamente la tabla. Además se simuló el circuito completo (no solo la tabla) sobre **2000 cadenas de bits aleatorias** (largo 1 a 15): para cada cadena se calcularon independientemente, con un buscador de subcadenas de Python (`str.find` con solapamiento), las posiciones donde termina una ocurrencia de "011", y se comparó contra las posiciones donde el circuito simulado (arrancando en  $S_0$ , aplicando  $D_1, D_0$  bit a bit) enciende  $Y=1$ . **Coincidencia exacta en las 2000 cadenas (0 fallas).**

Chequeo manual de una fila: estado  $S_2$  ( $q_1=1, q_0=0$ ),  $x=1$ .  $D_1 = q_1 \cdot q_0' + q_0 \cdot q_1' = 1 \cdot 1 + 0 \cdot 1 = 1$ .  $D_0 = x' + q_1 \cdot q_0' = 0 + 1 = 1$ . → próximo estado  $S_1 = S_3$  ✓ (coincide con la tabla:  $S_2, x=1 \rightarrow S_3$ ).

Guardar este ejercicio para comparar con el Ejercicio 3 (la versión Mealy del mismo problema): la versión Moore necesitó **4 estados**. Esa comparación es justamente el punto del Ejercicio 3.

## Ejercicio 2 ★ — FSM Moore: contador módulo 6 con indicador de múltiplo de 3

**Enunciado:** diseñar, con Flip-Flops T, un contador binario módulo 6 (recorre 0,1,2,3,4,5,0,...) que además tenga una salida  $Y$  (máquina de Moore) que se encienda ( $Y=1$ ) exactamente cuando el valor actual del contador es múltiplo de 3 (es decir, en los estados 0 y 3).

### PASO 1 — TABLA DE ESTADOS (3 BITS, $Q_2Q_1Q_0$ , CÓDIGOS 6 Y 7 NO SE USAN)

| Estado (dec) | $Q_2Q_1Q_0$ | próximo | $Y$ |
|--------------|-------------|---------|-----|
| 0            | 000         | 1 (001) | 1   |
| 1            | 001         | 2 (010) | 0   |
| 2            | 010         | 3 (011) | 0   |
| 3            | 011         | 4 (100) | 1   |
| 4            | 100         | 5 (101) | 0   |
| 5            | 101         | 0 (000) | 0   |

## PASO 2 Y 3 — TABLA DE EXCITACIÓN T (T=1 EXACTAMENTE CUANDO EL BIT CAMBIA ENTRE ESTADO Y PRÓXIMO ESTADO)

| Estado | $q_2q_1q_0$ | próximo | $T_2T_1T_0$ | $Y$ |
|--------|-------------|---------|-------------|-----|
| 0      | 000         | 001     | 001         | 1   |
| 1      | 001         | 010     | 011         | 0   |
| 2      | 010         | 011     | 001         | 0   |
| 3      | 011         | 100     | 111         | 1   |
| 4      | 100         | 101     | 001         | 0   |
| 5      | 101         | 000     | 101         | 0   |

Los códigos 110 y 111 (6 y 7) no ocurren nunca → se usan como *don't care* en el Karnaugh.

## PASO 4 Y 5 — ECUACIONES MINIMIZADAS (KARNAUGH CON DON'T-CARES 110,111)

$$T_2 = q_0q_1 + q_0q_2 \quad T_1 = q_0q_2' \quad T_0 = 1 \quad Y = q_0q_1 + q_0'q_1'q_2'$$

## PASO 6 — CIRCUITO

3 FF T.  $T_0=1$  fijo (siempre invierte, como en cualquier contador binario simple).  $T_1$  y  $T_2$ : compuertas AND/OR según las ecuaciones.  $Y$ : un OR de dos AND, tomado combinacionalmente del estado (Moore).

### ✓ VERIFICACIÓN

Se generaron los 8 códigos posibles de 3 bits, se marcaron los 6 estados válidos con su  $T_i$  exacto (por diferencia XOR bit a bit entre estado y próximo estado) y los 2 códigos no usados como *don't care*; SOPform minimizó  $T_2, T_1, T_0, Y$  usando esos comodines. Se evaluaron las 4 ecuaciones en los 6 estados válidos y coincidieron exactamente con la tabla (0 fallas). Chequeo manual: estado 3 (011).

$$T_2 = q_0q_1 + q_0q_2 = 1 \cdot 1 + 1 \cdot 0 = 1. \quad T_1 = q_0q_2' = 1 \cdot 0 = 0. \quad T_0 = 1.$$

$$Q_{n+1} = Q_n \oplus T: \quad q_2: 0 \oplus 1 = 1, \quad q_1: 1 \oplus 0 = 1, \quad q_0: 1 \oplus 0 = 1 \rightarrow \text{próximo} = 100 = 4 \checkmark.$$

$$Y = q_0q_1 + q_0'q_1'q_2' = 1 \cdot 1 + 0 = 1 \checkmark \quad (3 \text{ es múltiplo de } 3).$$

## Ejercicio 3 ★★ — FSM Mealy: mismo detector de “011” con solapamiento (comparar con el Ejercicio 1)

**Enunciado:** resolver el mismo problema del Ejercicio 1 (detectar “011” con solapamiento) pero con el enfoque de máquina de Mealy (la salida puede depender de estado y entrada actual simultáneamente). Usar FF D.

## PASO 1 — TABLA DE ESTADOS (MEALY NECESITA UN ESTADO MENOS)

- **A:** no hay prefijo útil.
- **B:** visto “0”.

- **C**: visto “01”.

No hace falta un cuarto estado “ya completé 011” porque en Mealy la salida se puede emitir **en la transición misma** que completa el patrón, sin necesitar un estado separado solo para “recordar que se emitió  $Y=1$  el ciclo pasado” (eso es justamente lo que obligaba a Moore a tener S3).

| Estado | $x=0 \rightarrow (\text{próximo}, Y)$ | $x=1 \rightarrow (\text{próximo}, Y)$                                                            |
|--------|---------------------------------------|--------------------------------------------------------------------------------------------------|
| A      | (B, 0)                                | (A, 0)                                                                                           |
| B      | (B, 0)                                | (C, 0)                                                                                           |
| C      | (B, 0)                                | (A, 1) — completa “011”, y como el último bit es “1” (no es prefijo de “011”) vuelve directo a A |

## PASO 2 Y 3 — CODIFICACIÓN (A=00, B=01, C=10; CÓDIGO 11 NO SE USA) Y TABLA DE TRANSICIONES

| Estado | $q_1q_0$ | $x$ | próximo | $D_1D_0$ | $Y$ |
|--------|----------|-----|---------|----------|-----|
| A      | 00       | 0   | B       | 01       | 0   |
| A      | 00       | 1   | A       | 00       | 0   |
| B      | 01       | 0   | B       | 01       | 0   |
| B      | 01       | 1   | C       | 10       | 0   |
| C      | 10       | 0   | B       | 01       | 0   |
| C      | 10       | 1   | A       | 00       | 1   |

## PASO 4 Y 5 — ECUACIONES (KARNAUGH DE 3 VARIABLES, CON DON'T-CARE EN $q_1q_0=11$ )

$$D_1 = q_0 \oplus x \quad D_0 = x' \quad Y = q_1 x$$

## PASO 6 — CIRCUITO

2 FF D.  $D_1$ : un solo AND ( $q_0 \cdot x$ ).  $D_0$ : un NOT.  $Y$ : un solo AND ( $q_1 \cdot x$ ) — mucho más simple que la  $Y$  del Ejercicio 1 porque acá **es combinatorial pura, no depende de haber “llegado” a un estado dedicado.**

## ✓ VERIFICACIÓN

Igual método que el Ejercicio 1: simulación sobre 2000 cadenas aleatorias, comparando contra las posiciones de fin-de-ocurrencia de “011” con solapamiento calculadas con `str.find`. **0 fallas.** Las ecuaciones se verificaron además evaluándolas en las 6 filas válidas de la tabla (0 fallas).



**Comparación explícita Moore vs. Mealy (el punto central de este ejercicio):** el Ejercicio 1 (Moore) necesitó **4 estados** (S0,S1,S2,S3) porque tuvo que dedicar un estado completo (S3) solo para “recordar, durante todo un ciclo de reloj, que se acaba de completar el patrón” y así poder mostrar  $Y=1$  sin mirar la entrada actual. Este ejercicio (Mealy), sobre el **mismo patrón**, resuelve el problema con **3 estados** (A,B,C) porque la salida se calcula en el instante de la transición (arco “C,  $x=1 \rightarrow A, Y=1$ ”), sin necesitar ese estado extra. Esto ilustra la regla general de la Guía M4 (sección 8.5): “Moore suele necesitar 1 estado extra por cada ‘versión con distinta salida’ del mismo estado Mealy” — acá, literalmente, S3(Moore) es la fusión de “estar en C y haber recibido  $x=1$ ” que en Mealy no requiere estado propio.

## Ejercicio 4 ★★★ — FSM Mealy: detector de “1001” sin solapamiento

**Enunciado:** con FF D, diseñar un detector de la secuencia “1001” que **no permita solapamiento**: una vez detectado el patrón, el circuito debe “reiniciarse” y empezar a buscar desde cero (a diferencia de los Ejercicios 1 y 3, que sí permiten solapamiento). Usar el enfoque de Mealy.

### PASO 1 — TABLA DE ESTADOS

- **S0:** nada útil reconocido.
- **S1:** visto “1”.
- **S2:** visto “10”.
- **S3:** visto “100”.

| Estado | $x=0 \rightarrow$ (próximo, Y)                           | $x=1 \rightarrow$ (próximo, Y)                                                                            |
|--------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| S0     | (S0, 0)                                                  | (S1, 0)                                                                                                   |
| S1     | (S2, 0)                                                  | (S1, 0) — un “1” repetido no rompe el prefijo “1”                                                         |
| S2     | (S3, 0)                                                  | (S1, 0) — “101”: el “1” final es un nuevo posible comienzo                                                |
| S3     | (S0, 0) — “1000”: se rompe todo, sin ningún prefijo útil | (S0, 1) — completa “1001”; <b>sin solapamiento</b> vuelve a S0 (no a S1, aunque el bit que llega sea “1”) |

Esta es la diferencia de tolerancia cero frente a los Ejercicios 1 y 3: en S3 con  $x=1$  se completa el patrón y la especificación pide **sin solapamiento**, así que el próximo estado es **S0** (se descarta por completo lo reconocido, incluso el “1” que acaba de llegar), no S1 como hubiera sido con solapamiento permitido. Confundir esto es el error más común al pasar de un detector con solapamiento a uno sin solapamiento.

## PASO 2 Y 3 — CODIFICACIÓN (S0=00, S1=01, S2=10, S3=11) Y TABLA DE TRANSICIONES

| Estado | $q_1 q_0$ | x | próximo | $D_1 D_0$ | Y |
|--------|-----------|---|---------|-----------|---|
| S0     | 00        | 0 | S0      | 00        | 0 |
| S0     | 00        | 1 | S1      | 01        | 0 |
| S1     | 01        | 0 | S2      | 10        | 0 |
| S1     | 01        | 1 | S1      | 01        | 0 |
| S2     | 10        | 0 | S3      | 11        | 0 |
| S2     | 10        | 1 | S1      | 01        | 0 |
| S3     | 11        | 0 | S0      | 00        | 0 |
| S3     | 11        | 1 | S0      | 00        | 1 |

## PASO 4 Y 5 — ECUACIONES (KARNAUGH DE 3 VARIABLES, SIN DON'T-CARES — SE USAN LOS 4 ESTADOS)

$$D_1 = q_0 q_1' x' + q_1 q_0' x \quad D_0 = q_1 q_0' + q_1' x \quad Y = q_1 q_0 x$$

## PASO 6 — CIRCUITO

2 FF D + compuertas AND/OR/NOT según las ecuaciones anteriores.

### ✓ VERIFICACIÓN

Se simularon **3000 cadenas aleatorias** (largo 1 a 20). Para cada una se calculó, con un contador de ocurrencias **sin solapamiento** (`str.find`) avanzando el puntero al final de cada coincidencia, no un carácter, como en los ejercicios 1 y 3), el conjunto de posiciones donde termina una detección válida, y se comparó contra las posiciones donde el circuito simulado enciende  $Y=1$ . **0 fallas**. Las ecuaciones también se verificaron evaluándolas en las 8 filas de la tabla (0 fallas).

Comparación de tamaño con una hipotética versión Moore del mismo patrón “1001”: necesitaría un quinto estado dedicado a “acabo de completar 1001” (igual que S3 en el Ejercicio 1 frente a C en el Ejercicio 3) — **5 estados Moore** contra los **4 estados Mealy** usados acá. El patrón se repite: cada vez que la salida de una transición Mealy “hacia atrás” (a un estado con historia distinta de cero, como S0 en vez de un estado “neutro”) tiene una salida distinta según desde qué estado se llegó, Moore necesita separar esos casos en estados nuevos.

## Ejercicio 5 ★★★ — Síntesis completa con FF JK: contador BCD (0 a 9)

**Enunciado:** diseñar, con Flip-Flops JK, un contador que recorra los dígitos BCD en binario natural: 0,1,2,...,9,0,... (4 bits, códigos 10 a 15 nunca deben aparecer en operación normal).

**PASO 1 — TABLA DE TRANSICIONES (10 ESTADOS VÁLIDOS; 6 CÓDIGOS NO USADOS → DON'T CARE)**

| Estado | \$q_3q_2q_1q_0\$ | próximo |
|--------|------------------|---------|
| 0      | 0000             | 0001    |
| 1      | 0001             | 0010    |
| 2      | 0010             | 0011    |
| 3      | 0011             | 0100    |
| 4      | 0100             | 0101    |
| 5      | 0101             | 0110    |
| 6      | 0110             | 0111    |
| 7      | 0111             | 1000    |
| 8      | 1000             | 1001    |
| 9      | 1001             | 0000    |

**PASO 2 — TABLA DE EXCITACIÓN JK (APLICANDO LA TABLA DE EXCITACIÓN DE LA GUÍA M4 SECCIÓN 5.2 A CADA BIT, FILA POR FILA)**

| Estado        | \$J_3K_3\$ | \$J_2K_2\$ | \$J_1K_1\$ | \$J_0K_0\$ |
|---------------|------------|------------|------------|------------|
| 0 (0000→0001) | 0,X        | 0,X        | 0,X        | 1,X        |
| 1 (0001→0010) | 0,X        | 0,X        | 1,X        | X,1        |
| 2 (0010→0011) | 0,X        | 0,X        | X,0        | 1,X        |
| 3 (0011→0100) | 0,X        | 1,X        | X,1        | X,1        |
| 4 (0100→0101) | 0,X        | X,0        | 0,X        | 1,X        |
| 5 (0101→0110) | 0,X        | X,0        | 1,X        | X,1        |
| 6 (0110→0111) | 0,X        | X,0        | X,0        | 1,X        |
| 7 (0111→1000) | 1,X        | X,1        | X,1        | X,1        |
| 8 (1000→1001) | X,0        | 0,X        | 0,X        | 1,X        |
| 9 (1001→0000) | X,1        | 0,X        | 0,X        | X,1        |

Más los 6 códigos 10-15, que se dejan como *don't care* completo (X en las 8 entradas J,K) porque nunca se alcanzan si el diseño y el reset son correctos.

### PASO 3 Y 4 — KARNAUGH DE 4 VARIABLES PARA CADA $J_i, K_i$ (APROVECHANDO TODOS LOS X COMO COMODINES)

$J_3 = q_0 q_1 q_2 \quad K_3 = q_0$   
 $J_2 = q_0 q_1 \quad K_2 = q_0 q_1$   
 $J_1 = q_0 q_3 \quad K_1 = q_0$   
 $J_0 = 1 \quad K_0 = 1$

### PASO 5 — CIRCUITO

4 FF JK.  $J_0 = K_0 = 1$  (el bit menos significativo siempre alterna — es el mismo patrón que un contador binario normal, coherente con la Guía M4 sección 6.6). El resto son AND de 2 o 3 entradas conectados a  $J_i, K_i$  según las ecuaciones.

### ✓ VERIFICACIÓN

Se generó la tabla de excitación JK de las 10 filas válidas invirtiendo la ecuación característica fila por fila (mismo método de la tabla de excitación JK de la Guía M4, sección 5.2), se agregaron los 6 códigos no usados como don't-care total, y `SOPform` minimizó cada  $J_i, K_i$  con esos comodines. Para verificar, se tomaron las 8 ecuaciones resultantes y, para cada uno de los 10 estados válidos, se evaluó  $Q_{n+1} = JQ' + K'Q$  (la ecuación característica JK) con los  $J, K$  calculados — el resultado coincidió **exactamente** con el próximo estado esperado en las 10 filas (0 fallas). Chequeo manual: estado 9 (1001) → 0 (0000).  $J_3 = q_0 q_1 q_2 = 1 \cdot 0 \cdot 0 = 0$ ,  $K_3 = q_0 = 1 \rightarrow q_3: 1 \rightarrow \overline{1} = 0$ .  $J_0 = 1, K_0 = 1 \rightarrow q_0: 1 \rightarrow \overline{1} = 0$ .

Nota de método: a diferencia de los contadores con FF D de la Guía M4 (secciones 6.2 a 6.5), acá cada entrada de FF (J y K) se obtiene **de a una** a partir de la tabla de excitación de la Parte 5.2, no copiando directamente el próximo estado. Es más trabajo de tabla, pero típicamente da ecuaciones más simples gracias a los comodines “X” — compárese, por ejemplo,  $J_0 = K_0 = 1$  (una constante) contra lo que hubiera dado  $D_0$  con FF D para el mismo contador (una expresión no trivial, como en la Guía M4 sección 6.2).

## Ejercicio 6 ★★ — Síntesis completa con FF D: tabla de transición dada directamente

**Enunciado:** un circuito secuencial de 2 bits de estado ( $q_1 q_0$ ) y una entrada  $x$  está especificado por la siguiente tabla de transición y salida (ya codificada — no hace falta inventar un diagrama de estados, solo completar el diseño). Diseñarlo con Flip-Flops D.

| $q_1q_0$ (estado) | x | próximo $q_1q_0$ | Y |
|-------------------|---|------------------|---|
| 00                | 0 | 01               | 0 |
| 00                | 1 | 10               | 1 |
| 01                | 0 | 11               | 0 |
| 01                | 1 | 00               | 1 |
| 10                | 0 | 00               | 0 |
| 10                | 1 | 11               | 1 |
| 11                | 0 | 10               | 1 |
| 11                | 1 | 01               | 0 |

Este ejercicio practica específicamente los pasos 4-6 de la metodología (Guía M4, sección 8.3) de forma aislada: no hay que inventar el diagrama de estados a partir de un enunciado en palabras (eso ya está dado), solo pasar de la tabla a las ecuaciones y al circuito — es la parte que más rinde practicar de memoria muscular porque es mecánica pero fácil de errar bajo presión de tiempo.

### PASO 1 — ECUACIONES DE EXCITACIÓN (FF D: $D_1 = \text{PRÓXIMO VALOR DE } q_1$ DIRECTAMENTE, SE TOMA TAL CUAL DE LA TABLA)

Los 4 estados están todos usados (2 bits = 4 códigos, sin códigos sobrantes) → no hay don't-cares.

### PASO 2 — KARNAUGH DE 3 VARIABLES ( $q_1, q_0, x$ ) PARA $D_1$ , $D_0$ E $Y$

$D_1 = q_0 x' + q_0' x \quad (\text{XOR entre } q_0 \text{ y } x)$   
 $D_0 = q_1 x + q_1' x' \quad (\text{XNOR entre } q_1 \text{ y } x)$   
 $Y = q_0' x + q_1' x + q_0 q_1 x$

### PASO 3 — CIRCUITO

2 FF D.  $D_1$  y  $D_0$  se arman directamente con una compuerta XOR y una XNOR respectivamente (o su equivalente en AND/OR/NOT si no hay XOR/XNOR disponible).  $Y$ : 3 AND de 2 entradas + 1 OR de 3 entradas.

### ✓ VERIFICACIÓN

Se tomaron las 8 filas de la tabla tal como están dadas en el enunciado (sin pasar por ningún diagrama intermedio), se armaron los minitérminos de  $D_1, D_0, Y$  y se minimizaron con `SOPform`. Se evaluaron las 3 ecuaciones resultantes en las 8 combinaciones de  $(q_1, q_0, x)$  y coincidieron exactamente con la tabla dada (0 fallas). Chequeo manual: fila  $q_1q_0=10$ ,  $x=1$ .  $D_1 = q_0 x' + q_0' x = 0 + 1 \cdot 1 = 1$ .  
 $D_0 = q_1 x + q_1' x' = 1 \cdot 1 + 0 = 1$ . → próximo  $11$  ✓.  $Y = q_0' x + q_1' x + q_0 q_1 x = 1 + 0 + 0 = 1$  ✓.

## Ejercicio 7 ★★★ — Análisis: dado el circuito, hallar la tabla de estados y el diagrama

**Enunciado:** se tiene un circuito secuencial síncrono con dos Flip-Flops JK ( $Q_1$ ,  $Q_0$ , mismo reloj) y una entrada  $x$ , con la siguiente lógica combinacional de excitación:

$$J_1 = x \oplus q_0 \quad K_1 = q_0 \quad J_0 = x \quad K_0 = q_1$$

(es decir: una compuerta XOR entre  $x$  y  $q_0$  alimenta  $J_1$ ; la salida  $q_0$  se conecta directo a  $K_1$ ; la entrada  $x$  se conecta directo a  $J_0$ ; la salida  $q_1$  se conecta directo a  $K_0$  — no hay ninguna salida  $Y$  adicional, se pide solo la tabla de estados y el diagrama). Hallar la tabla de estados completa y el diagrama de transiciones.

### PASO 1 — EVALUAR $J, K$ PARA LAS 8 COMBINACIONES DE $(Q_1, Q_0, X)$

| $q_1q_0$ | $x$ | $J_1$ | $K_1$ | $J_0$ | $K_0$ | próximo $q_1q_0$ |
|----------|-----|-------|-------|-------|-------|------------------|
| 00       | 0   | 0     | 0     | 0     | 0     | 00               |
| 00       | 1   | 1     | 0     | 1     | 0     | 11               |
| 01       | 0   | 1     | 1     | 0     | 0     | 11               |
| 01       | 1   | 0     | 1     | 1     | 0     | 01               |
| 10       | 0   | 0     | 0     | 0     | 1     | 10               |
| 10       | 1   | 1     | 0     | 1     | 1     | 11               |
| 11       | 0   | 1     | 1     | 0     | 1     | 00               |
| 11       | 1   | 0     | 1     | 1     | 1     | 00               |

Cada fila se calculó aplicando la ecuación característica JK,  $Q_{n+1} = JQ' + K'Q$ , por separado a  $q_1$  (con  $J_1, K_1$ ) y a  $q_0$  (con  $J_0, K_0$ ).

### PASO 2 — TABLA DE ESTADOS EQUIVALENTE (RENOMBRANDO $00=A$ , $01=B$ , $10=C$ , $11=D$ )

| Estado | $x=0 \rightarrow$ próximo | $x=1 \rightarrow$ próximo |
|--------|---------------------------|---------------------------|
| A (00) | A                         | D                         |
| B (01) | D                         | B                         |
| C (10) | C                         | D                         |
| D (11) | A                         | A                         |

### PASO 3 — DIAGRAMA DE TRANSICIONES (DESCRIPCIÓN) Y OBSERVACIÓN DE ALCANZABILIDAD

- $A \xrightarrow{0} A$  (auto-loop),  $A \xrightarrow{1} D$ .
- $B \xrightarrow{0} D$ ,  $B \xrightarrow{1} B$  (auto-loop).
- $C \xrightarrow{0} C$  (auto-loop),  $C \xrightarrow{1} D$ .
- $D \xrightarrow{0} A$ ,  $D \xrightarrow{1} A$  (ambas van a A).

Punto de examen frecuente en un análisis: verificar **alcanzabilidad** desde el estado de reset. Si el circuito arranca en A (000... o sea  $q_1q_0=00$ , típico tras un Clear), los únicos estados que se visitan alguna vez son **A y D** ( $A \leftrightarrow D$  es un ciclo cerrado:  $A-0 \rightarrow A$ ,  $A-1 \rightarrow D$ ,  $D-0 \rightarrow A$ ,  $D-1 \rightarrow A$  — nunca sale de  $\{A,D\}$ ). Los estados **B y C son inalcanzables** desde el reset, aunque están perfectamente definidos en la tabla (el análisis siempre debe completar las 8 filas igual, sin asumir alcanzabilidad de antemano: si por un glitch, un valor inicial de Flip-Flop indefinido, o un fallo transitorio el circuito cayera en B o C, seguiría teniendo un comportamiento bien definido — con las ecuaciones de arriba, B es un estado que retiene su valor en  $x=1$  pero escapa en  $x=0$ , y C es simétrico). Reportar la alcanzabilidad es parte de una respuesta completa de análisis.

#### ✓ VERIFICACIÓN

Se evaluaron las 4 ecuaciones combinacionales ( $J_1, K_1, J_0, K_0$ ) para las 8 combinaciones de  $(q_1, q_0, x)$  con Python, se aplicó la ecuación característica JK por separado a cada bit, y se construyó la tabla de arriba mecánicamente (sin asumir ningún resultado de antemano). Además se verificó la alcanzabilidad por búsqueda en anchura (BFS) desde el estado inicial  $A=00$ : el conjunto de estados alcanzables resultó exactamente  $\{A,D\}$ , confirmando que B y C nunca se visitan partiendo de un reset a 00.

## Preguntas teóricas rápidas tipo parcial

| # | Pregunta                                                                                                                                                                                             | Respuesta                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | ¿Cuándo conviene usar Moore en vez de Mealy, aunque Moore típicamente use más estados?                                                                                                               | Cuando la salida no debe cambiar hasta el próximo flanco de reloj (comportamiento más predecible/sincronizado, sin “glitches” combinacionales que se propaguen a la salida apenas cambia la entrada) — por ejemplo, si la salida maneja directamente un actuador externo donde un cambio a mitad de ciclo sería peligroso o inválido. Mealy conviene cuando se busca <b>minimizar la cantidad de estados</b> (y por lo tanto de Flip-Flops) y se puede tolerar que la salida reaccione de inmediato a la entrada actual. |
| 2 | En una máquina de Mealy, ¿puede la salida cambiar sin que haya flanco de reloj?                                                                                                                      | Sí, si la salida se calcula de forma puramente combinacional a partir de estado y entrada actual — un cambio de la entrada (sin esperar el flanco) puede hacer cambiar $Y$ de inmediato. En Moore esto no puede pasar: $Y$ solo cambia cuando cambia el estado, es decir, en el flanco.                                                                                                                                                                                                                                  |
| 3 | Dada la tabla de excitación JK, ¿qué valores de $J, K$ producen la transición $Q(t+1) = 1$ $\wedge$ $Q(t) = 1$ ?                                                                                     | $J=X$ (no importa), $K=0$ (obligatorio) — porque tanto “mantener” ( $J=0, K=0$ ) como “set” ( $J=1, K=0$ ) producen $Q_{n+1}=1$ partiendo de $Q=1$ ; lo único que no puede pasar es $K=1$ (eso resetearía o invertiría a 0).                                                                                                                                                                                                                                                                                             |
| 4 | Un contador de 10 estados (BCD) diseñado con FF JK: ¿por qué suele quedar más simple que el mismo contador con FF D?                                                                                 | Porque la tabla de excitación JK introduce comodines “X” en casi todas las filas (dos de las cuatro combinaciones $J, K$ producen cada transición), lo que da más libertad al Karnaugh para agrupar y simplificar — con FF D no hay comodines “naturales” salvo los que vengan de estados no usados.                                                                                                                                                                                                                     |
| 5 | En un ejercicio de <b>análisis</b> (circuito dado, hay que hallar la tabla), ¿alcanza con reportar solo los estados alcanzables desde el reset?                                                      | No — hay que completar la tabla de estados para <b>todas</b> las combinaciones posibles de estado y entrada, existan o no en el camino normal desde el reset. Reportar además cuáles son alcanzables (por ejemplo con un BFS desde el estado inicial) es un plus que se suele pedir explícitamente, pero no reemplaza la tabla completa.                                                                                                                                                                                 |
| 6 | ¿Por qué el detector de “1001” sin solapamiento (Ejercicio 4) tiene una transición distinta a la del detector de “011” con solapamiento (Ejercicios 1 y 3) en el estado donde se completa el patrón? | Porque “sin solapamiento” exige descartar por completo el prefijo reconocido al detectar el patrón, incluso si el último bit recibido podría haber sido el comienzo de una nueva ocurrencia — por eso ese caso va al estado “neutro” (sin ningún bit útil recordado) en vez de al estado que representa “ya vi el primer bit del próximo posible patrón”, que es lo que sí hace la versión con solapamiento.                                                                                                             |

## Resumen ultra-rápido de este documento

- **Moore vs Mealy, mismo patrón:** el mismo detector puede necesitar 1 estado más en Moore que en Mealy — el estado extra existe únicamente para “recordar durante un ciclo entero” que la salida debe estar en 1, algo que Mealy resuelve con una salida sobre el arco de transición.



- **Solapamiento vs sin solapamiento:** la única diferencia de diseño está en la transición que sale del estado donde se completa el patrón — con solapamiento, esa transición reutiliza el sufijo del patrón recién completado como posible nuevo comienzo; sin solapamiento, siempre vuelve al estado inicial “neutro”.
- **Síntesis con JK vs D:** JK da más comodines (tabla de excitación con X) → Karnaugh más simple, a costa de tener que armar 2 ecuaciones por bit de estado en vez de 1.
- **Análisis (circuito → tabla):** siempre completar las 8 (o  $2^n$ ) filas evaluando la lógica combinacional dada, sin asumir de antemano cuáles son alcanzables; reportar la alcanzabilidad aparte.
- Todo lo anterior sigue siendo, en el fondo, la misma metodología de 6 pasos de la Guía Temática M4 (sección 8.3) — estos ejercicios solo cambian la especificación (otro patrón, otro tipo de FF, tabla dada directamente, o circuito dado directamente) para practicar el método completo, no un ejercicio de memoria.